PolicyCenter Simulator

Complete project explanation and architecture

Insurance policy administration simulator for Personal Auto and Homeowners, modelled on the way Guidewire PolicyCenter works. Built in Node.js with a browser user interface.

Repository: github.com/Sreenu123196/Policy_Center (branch main)
Last commit: 3eb15ff, "Add profitability, Insights dashboard, chat language picker and faster voice"
Document date: 17 October 2026

Contents

1. Summary
2. Purpose and scope
3. Architecture diagram
4. Components and responsibilities
5. Business flows (diagrams)
6. Domain model and business rules
7. Roles and access
8. Sign-in and security
9. Data storage
10. Chatbot, voice and customer messages
11. Finance and Insights
12. User interface
13. API reference
14. Configuration, running and testing
15. Known limitations and next steps

1. Summary

PolicyCenter Simulator is a working demonstration of a policy administration system for property and casualty insurance. It runs on one computer with a Node.js server, a browser application and a local data file. It is not a copy of Guidewire PolicyCenter; it follows the same business vocabulary (accounts, submissions, quotes, binding, policy changes, renewals, cancellations, underwriting issues, billing and claims) so that the workflows can be understood and demonstrated.

An agent creates a customer account and submits an application; the rating engine prices it from effective-dated rate books; underwriting rules raise issues that a person with the right authority approves; binding creates the policy and its billing; the policy can then be changed, renewed, cancelled or reinstated. Claims, customer tickets with service-level timers, sales analytics, customer messages (email and WhatsApp), a profitability settlement report, an Insights dashboard and a Claude-powered assistant that answers in English, Telugu and Hindi are built on the same data.

Access is role-based. Four roles (Agent, Underwriter, Insurance Executive, UW Manager) see different data and actions, and every person signs in with their own username and password. The system has 69 HTTP routes and 103 automated tests; all 103 pass at the last commit.

2. Purpose and scope

Purpose

- Show how a policy administration system is structured and how a policy moves through its lifecycle.
- Give people a safe place to learn the workflows, roles and reports without touching real customer data.
- Demonstrate newer features on top of the core: a voice and text assistant, customer messaging and analytics.

Products

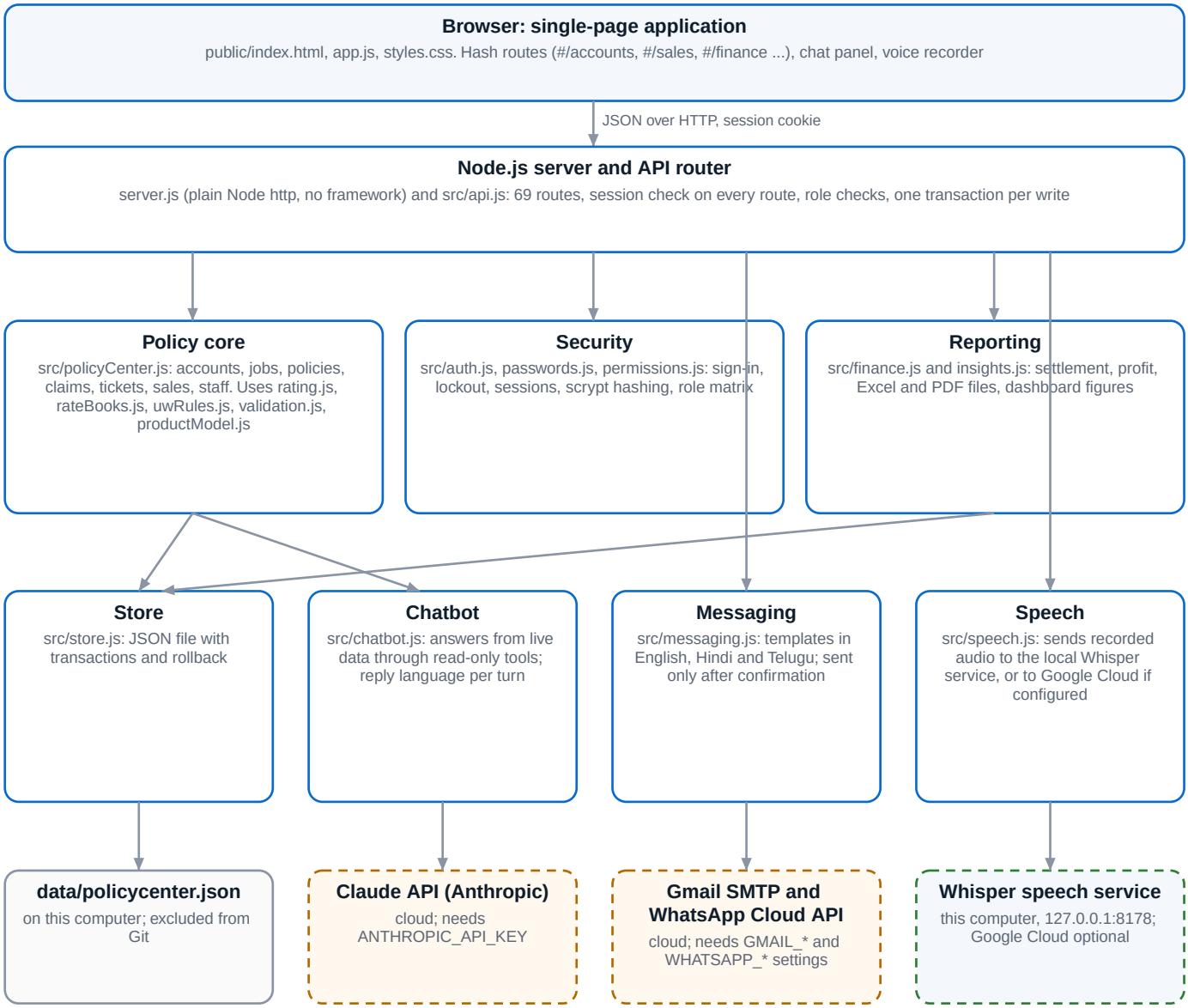
- Personal Auto: drivers, vehicles, coverages and state rating (Texas, Illinois, Florida, Ohio, California and New York in the demo rate books).
- Homeowners: dwelling and property details, forms HO-3 and HO-5, and replacement-cost estimates.

Out of scope

- Real money movement, payment processing, or connection to a live carrier system.
- Actuarial accuracy: the rate books and underwriting rules are simplified demonstration values.
- Production security hardening (see section 15).

3. Architecture diagram

Solid boxes run on this computer. Dashed boxes are outside services that the system calls over the internet, when they are configured.



Request path: the browser sends a request; the API router identifies the signed-in person from the session cookie and checks that the route is allowed for their role; a write runs inside one store transaction, so a failed change is rolled back. The chatbot and the messaging service use the same role rules as the screens, so they cannot show or send anything the person could not see there.

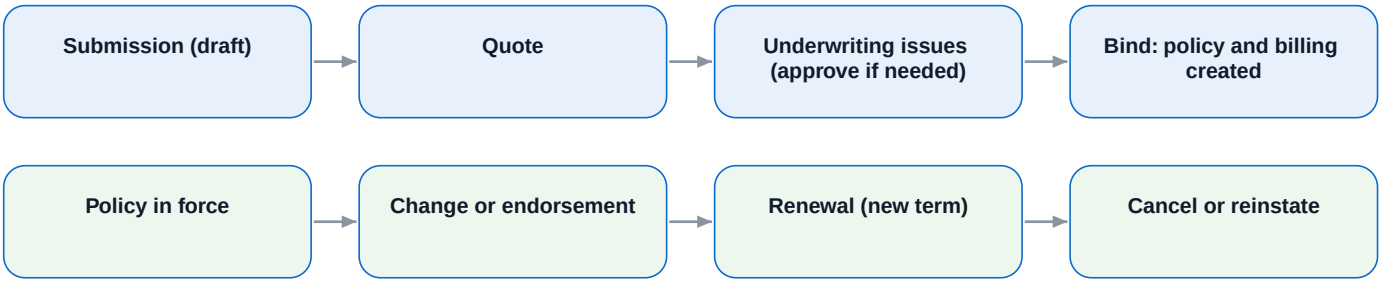
4. Components and responsibilities

File	Responsibility
server.js	Starts the server on port 3000 (PORT overrides it); creates the store, services and API; serves the browser files.
src/api.js	Maps each URL to a handler, checks the session on every route, enforces the first-password change, returns JSON or streams (chat, Excel, PDF).
src/policyCenter.js	Core business logic: accounts, submissions, quotes, binding, policy changes, renewals, cancellations, reinstatement, claims, tickets, sales, follow-ups, staff.
src/rating.js, rateBooks.js	Premium calculation from effective-dated rate books, with a factor worksheet that explains each factor.
src/uwRules.js	Underwriting rules that raise issues; each issue needs approval at a set authority level.
src/validation.js, productModel.js	Field checks and the product model (lines, coverages, limits, forms, terms) for Personal Auto and Homeowners.
src/auth.js, passwords.js	Sign-in with lockout after repeated failures, 8-hour sessions, scrypt password hashing, temporary passwords.
src/permissions.js	Single place that decides what each role may see and do.
src/store.js	Loads and saves the JSON data file; runs each write as one transaction; migrates older data.
src/finance.js	Monthly settlement: premium, commission, expenses, net profit and trend; Excel and PDF output.
src/insights.js	Insights figures: policies sold and profit by month, product, producer and salesperson.
src/chatbot.js	Assistant: Claude model with read-only tools over the live data; offline lookups when no key is set.
src/speech.js	Turns recorded audio into text: local Whisper (default) or Google Cloud Speech.
src/messaging.js	Customer messages by email and WhatsApp, with templates and a confirmation step before sending.
src/seed.js, scripts/	Demo data; generate-test-data.js (100 accounts); reset-manager.js (recovery); whisper_server.py (local speech service).
public/	Browser application: index.html, app.js (all pages and actions), styles.css.

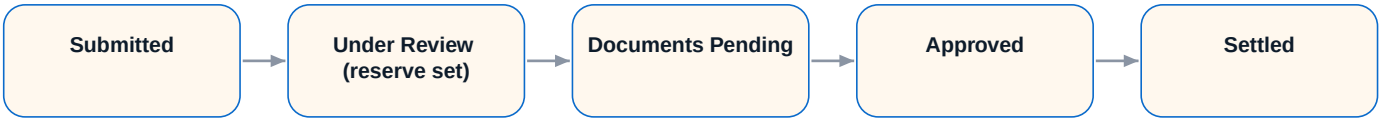
5. Business flows (diagrams)

Policy lifecycle

A submission is a job. Once bound it becomes a policy version. Later jobs (change, renewal, cancellation, reinstatement) add new versions to the same policy.

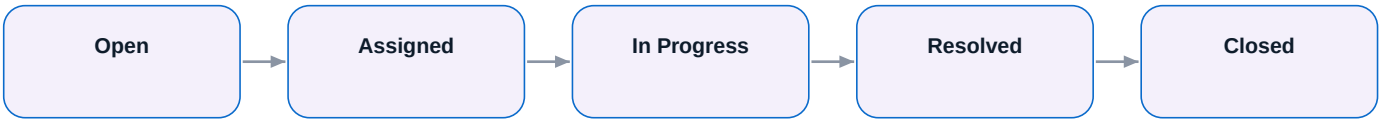


Claim stages



A claim can also go from Submitted or Under Review to Rejected. A claim cannot move to Approved without passing Under Review. Customer messages are sent in English, Hindi or Telugu when each stage changes.

Ticket stages and response times



Response deadlines are Urgent 4 hours, High 24 hours, Normal 48 hours and Low 120 hours. Premium customers (total current premium of 3,000 or more including tax) are placed first in the ticket queue. A Resolved ticket can go back to In Progress.

6. Domain model and business rules

Area	What the system does
Accounts	Customer (named insured) with address, date of birth, contact details and the producer code of the selling agency. Agents see only their own agency's accounts.
Submissions and rating	Drivers, vehicles, dwelling and coverages are entered on a job. The rating engine uses the rate book in force on the effective date and records each factor in a worksheet.
Underwriting	Rules raise issues (for example, a driver with an accident, or a high replacement-cost ratio). An issue must be approved by a person with the required authority before the job can be bound.
Policies	Each policy keeps its versions and terms. Changes, renewals and cancellations create new versions, so the history is kept.
Billing	Binding and changes create invoices. A return or cancellation creates a credit. The billing report shows invoices by status and product.
Claims	Opened against a policy with a loss date and type. Reserves are set by underwriters or managers. Payments are recorded against approved claims. Stages follow the transitions in section 5.
Sales	A sale is a policy whose first job was a submission, counted on the day it was bound. Sales analytics group sales by day, month or year and by salesperson.
Lapsing and follow-up	A policy is lapsing when its term ends within 60 days and it has not renewed; lapsed when it ended within the last 90 days. Follow-ups record the channel (Phone, Email, SMS, Visit, Letter) and the result.
Reports	Claims report, billing report, sales analytics, the settlement report and the Insights dashboard. Each shows only the figures its role may see.

Roles and access

Role	Sees	Can do
Agent	Own agency's accounts, policies and sales; own follow-ups.	Create accounts, submit and quote, bind, make changes, log follow-ups, use the chat for their own data.
Underwriter	All accounts and policies, rating factors, underwriting notes, claim reserves, claims and billing reports for the portfolio.	Approve underwriting issues, set claim reserves and stages, review submissions.
Insurance Executive (limited)	Limited data: portfolio summary and sales figures for all salespeople, but not billing or rating detail.	Read-only: views sales analytics and lapsing lists but cannot log follow-ups; no billing report and no settlement figures.
UW Manager	Everything, including billing, the settlement report, profit, Insights and staff accounts.	Create and reset staff accounts, set the commission rate, enter expenses, run renewal batches, reset demo data.

The role rules are in one file (src/permissions.js). The screens, the API and the chatbot all use the same rules, so a person cannot see through one channel what they are refused in another.

8. Sign-in and security

- Passwords are stored as scrypt hashes with a random salt (src/passwords.js). Plain-text passwords are never stored or logged.
- The first start asks the first person to create the UW Manager account (the first-setup screen). No shared default password exists.
- The UW Manager creates the other staff accounts and sets their temporary passwords; each person must choose their own password at first sign-in.
- After 5 failed sign-ins the account is locked for 15 minutes.
- A session lasts 8 hours. The session token is an HttpOnly cookie with SameSite=Strict, so page scripts cannot read it.
- Every API route except the sign-in routes needs a valid session. A person with a temporary password can only change it until they have done so.
- API keys and passwords are read from environment variables and are never written into code, the data file or the repository.
- Sessions are kept in memory: restarting the server signs everyone out.

9. Data storage

All data is kept in one JSON file, data/policycenter.json, which is excluded from Git. The store (src/store.js) holds accounts, policies, jobs, activities, invoices, documents, events, claims, tickets, follow-ups, messages, finance settings and staff. Each write runs inside a transaction: the change is applied to a copy and saved only if the whole operation succeeds, so a half-finished change cannot be left behind. Demo data is created on first start from src/seed.js; scripts/generate-test-data.js adds 100 accounts with fixed random numbers, so the same data can be recreated.

10. Chatbot, voice and customer messages

Chatbot

- Uses the Claude model set by CHAT_MODEL (default claude-haiku-5-5) through the Anthropic SDK. The model calls read-only tools over the live data and answers only from what those tools return.
- Every tool runs as the signed-in person, so the chatbot follows the same role rules as the screens.
- Reply language: a language picked in the chat header (Auto-detect, English, Telugu, Hindi) is used for the answer. With Auto-detect the reply follows the language of the question, including Hinglish and Tenglish.
- Without an API key the chatbot runs in offline mode: it answers simple lookups and summaries in English only, and says so.

Voice

- The browser records a 16 kHz mono clip while the mic is on; the clip goes to /api/speech with the picked language as a hint.
- Default engine: the local Whisper speech service (scripts/whisper_server.py, small model, about 2 to 4 seconds for a short question). Telugu and Hindi accuracy is better with the medium model (WHISPER_MODEL=medium), which is slower.
- Optional engine: Google Cloud Speech-to-Text v2, when a service-account file is configured. If neither is available, the browser's own recognition is used, in English only.

Customer messages

- Email through Gmail SMTP (GMAIL_USER and GMAIL_APP_PASSWORD) and WhatsApp through the Meta Cloud API (WHATSAPP_TOKEN and WHATSAPP_PHONE_ID).

- Templates in English, Hindi and Telugu for renewals, lapse notices and claim updates. A preview shows the exact text and recipient before sending; nothing is sent without confirmation.
- Each message is saved on the customer record with its channel and result.

11. Finance and Insights

The settlement report (UW Manager only) shows, for each month: billed premium (by the due date of each invoice, net of credits), commission (one percentage of premium, 15% by default and changeable), income (premium plus commission), expenses (Salaries, Office rent and Other costs, entered in the app) and net profit (income minus expenses). The default range is the last 12 months. The report downloads as an Excel workbook or a PDF.

The Insights dashboard (UW Manager only) shows policies sold, premium sold, income, expenses and profit for the chosen months; a monthly table with a profit bar; breakdowns by product, producer (agency) and salesperson with commission earned; a grid of policies per salesperson per month; and four pie charts (policies by product, premium by producer, policies by salesperson, and expenses by category). The charts are drawn in plain SVG with no outside script.

12. User interface

The browser application is one page that changes its content by the URL hash. Each page is drawn by a function in public/app.js:

Hash route	Page	Who
#/desktop	Desktop: work queue, activities, tickets and key figures	Everyone
#/accounts, #/account/:id	Account list; account detail with policies, messages and new submissions	Everyone (within their role)
#/policies, #/policy/:id, #/job/:id	Policy list; policy with versions and billing; the job wizard (drivers, vehicles, coverages, issues)	Everyone (within their role)
#/sales	Sales analytics, lapsing and lapsed policies, follow-up history	Agents (own); Insurance Executive (read only) and UW Manager (all)
#/insights	Insights dashboard with pie charts	UW Manager
#/finance	Monthly settlement, commission rate, expenses, Excel and PDF downloads	UW Manager
#/staff	Staff accounts: create, reset temporary password, activate or deactivate	UW Manager
#/admin	Administration: reference data and demo reset	UW Manager

The chat panel sits at the bottom right of every page. The menu shows Insights, Finance and Staff accounts only to the UW Manager.

13. API reference

All routes are under /api. Sign-in routes are public; all others need the session cookie. The main groups are listed below (69 routes in total).

Group	Routes
Sign-in	GET /auth/status, POST /auth/setup, POST /auth/login, POST /auth/logout, GET /auth/me, POST /auth/password
Staff	GET and POST /staff, POST /staff/:id/reset, POST /staff/:id/active
Reference and search	GET /meta, GET /desktop, GET /search, GET /ratebooks, GET /events, GET /activities, POST /activities/:id/complete
Accounts	GET and POST /accounts, GET and PUT /accounts/:id, GET /accounts/:id/messages
Jobs (submission and changes)	POST /jobs/submission, GET /jobs/:id, PUT /jobs/:id/period, POST /jobs/:id/quote, /bind, /withdraw, /decline, POST /jobs/:id/uw-issues/:key/approve
Policies and claims	GET /policies, GET /policies/:id, POST /policies/:id/change, /cancel, /reinstate, /renew, POST /batch/renewals, GET /claims, POST /claims/:id/stage
Tickets	GET and POST /tickets, GET /tickets/queue, GET /tickets/:id, POST /tickets/:id/assign, /status, /notes
Sales	GET /sales/analytics, GET /sales/lapsing, GET /sales/followups, GET and POST /sales/policies/:id/followups
Reports	GET /reports/claims, GET /reports/billing, GET /insights, GET /reports/profitability.xlsx and .pdf
Finance	GET /finance/report, GET and POST /finance/expenses, POST /finance/commission
Messages	GET /messages/status, POST /messages/preview, POST /messages/send
Assistant and voice	GET /chat/status, POST /chat, POST /chat/stream (live answer), GET /speech/status, POST /speech
Administration	POST /admin/reset (UW Manager; resets demo data)

14. Configuration, running and testing

Environment settings (set them in the terminal, never in the code)

Name	Purpose
PORT	Web port (default 3000).
DATA_FILE	Location of the data file (default data/policycenter.json).
ANTHROPIC_API_KEY, ANTHROPIC_WORKSPACE_ID	Chatbot access. Without them the chatbot runs in offline mode.
CHAT_MODEL	Claude model for the chatbot (default claude-haiku-5-5).
WHISPER_MODEL, WHISPER_BEAM	Speech model (default small) and decoding beam (default 1, faster).
GOOGLE_APPLICATION_CREDENTIALS, GOOGLE_CLOUD_PROJECT	Optional Google Cloud speech instead of Whisper.
GMAIL_USER, GMAIL_APP_PASSWORD	Email sending through Gmail. The app password is created in the Google account.
WHATSAPP_TOKEN, WHATSAPP_PHONE_ID	WhatsApp sending through the Meta Cloud API.

Running

- Install once with npm install. Start the app with npm start, then open http://localhost:3000.
- Speech (optional): run the Whisper service in its own window, using the .whisper Python environment and scripts/whisper_server.py.
- Lost UW Manager password: stop the app and run npm run reset-manager. A new temporary password is printed in that window only.
- Bulk test data: npm run generate creates 100 accounts and their policies.

Testing

npm test runs 103 automated tests in 15 files with the built-in Node test runner. They cover rating, the policy lifecycle, underwriting, claims, tickets, sales, customer messages, roles, sign-in and sessions, the chatbot (including language handling), speech, the Whisper service, finance and the Insights figures. All 103 pass at the last commit.

Test file	What it checks
rating.test.js, lifecycle.test.js	Premium calculation and the policy lifecycle from submission to renewal
sales.test.js, tickets.test.js	Sales analytics and lapsing; tickets, SLA and claim stages
roles.test.js, auth.test.js	Role matrix; sign-in, lockout, setup, staff accounts and session checks over HTTP
messaging.test.js	Templates in three languages and the confirmation step
chatbot.test.js, speech.test.js, whisper.test.js	Chatbot tools and reply language; speech routing and the Whisper service
finance.test.js, finance-api.test.js, insights.test.js	Settlement figures, downloads, role limits and Insights figures

15. Known limitations and next steps

Limitations today

- Single computer, single process, JSON file storage: suitable for demonstration and training, not for many users at once or for real customer data.
- Sessions are held in memory and are lost when the server restarts. There is no HTTPS, so the app must stay on localhost or behind a secure proxy.
- Rate books, underwriting rules and commission are simplified demonstration values.
- Telugu and Hindi speech accuracy with the default small model has not been tested with real recordings; the medium model is more accurate and slower.
- Email and WhatsApp sending needs your own Gmail and Meta credentials. Until they are set, messages can be previewed but not delivered.
- The Insights, Finance and chat pages were checked by tests and by parsing the code, not yet by a browser walk-through.

Planned, not yet built

- Customer portal: sign-in with policy number and date of birth, so customers can raise tickets and claims themselves.
- Claim document upload, stored on this computer, and automatic tickets from claims and billing.
- Automatic WhatsApp and email updates when a claim stage or ticket changes.
- Production hardening: a real database, persistent sessions, HTTPS and audit logs.

Repository

Source code: github.com/Sreenu123196/Policy_Center, branch main. Generated data, the Whisper Python environment and node_modules are excluded from Git.